UNIVERSITY ERP SYSTEM

PROJECT REPORT

TABLE OF CONTENTS

---

1. EXECUTIVE SUMMARY

The University Enterprise Resource Planning (ERP) System is a comprehensive software solution designed to streamline the administrative and academic processes of a higher education institution. In the modern educational landscape, the efficient management of student data, course scheduling, and academic grading is paramount. This project aims to digitize these core functions, replacing manual, paper-based workflows with a robust, centralized digital platform.

The system caters to three primary user roles: Administrators, Instructors, and Students. Administrators serve as the backbone of the system, managing the master data including user accounts, course catalogs, and academic schedules. Instructors are empowered to manage their specific course sections and perform grading activities with ease. Students are provided with a self-service portal to manage their course enrollments and view their academic progress.

Built using Java for the core application logic and MySQL/TiDB for data persistence, the system emphasizes data integrity, security, and ease of use. Key features include a secure authentication system using BCrypt hashing, a dynamic course registration engine with capacity and deadline checks, and an automated grading system. This report details the lifecycle of the project from initial analysis through design, implementation, and testing, highlighting the architectural decisions and technical challenges addressed during development.

---

2. INTRODUCTION

2.1 BACKGROUND

Educational institutions generate vast amounts of data daily, ranging from student admissions details to complex grading hierarchies. Managing this data manually is error-prone, time-consuming, and inefficient. An ERP system integrates these various functions into a single system to facilitate information flow between all stakeholders.

2.2 PROBLEM STATEMENT

Prior to this system, the university faced several challenges:

- Data Redundancy: Student information was often duplicated across different departments (e.g., admissions vs. academic departments).

- Inefficient Scheduling: Course sectioning and room allocation were done manually, leading to conflicts and overbooking.

- Grading Errors: Manual calculation of final grades from various components (quizzes, midterms) often resulted in calculation errors.

- Lack of Transparency: Students had limited visibility into their real-time academic standing and course availability.

2.3 OBJECTIVES

The primary objectives of this project are:

- To develop a centralized database for storing all academic data.

- To automate the course registration process, enforcing rules such as prerequisites, capacity limits, and deadlines.

- To provide a secure interface for instructors to input grades and automatically calculate final GPAs.

- To ensure data security through role-based access control (RBAC) and encrypted authentication.

2.4 SCOPE OF THE PROJECT

The current version of the University ERP System covers the following modules:

- User Authentication & Authorization (Login/Logout).

- Academic Administration (Course/Section creation).

- Student Enrollment (Add/Drop courses).

- Grading System (Component-based scoring and final grade computation).

- System Maintenance (Backup/Restore and Maintenance Mode).

It does not currently cover Financial/Fee management, Library management, or Hostel allocation, which are slated for future phases.

---

3. SYSTEM ANALYSIS

3.1 USER ROLES AND CHARACTERISTICS

The system identifies three distinct actors, each with specific privileges:

A. ADMINISTRATOR

- Characteristics: High technical literacy, trusted personnel.
- Responsibilities: System configuration, user onboarding, master data management.
- Access: Full access to all modules, including direct database backup/restore functions.

B. INSTRUCTOR

- Characteristics: Academic staff, focused on teaching and assessment.
- Responsibilities: Managing assigned sections, grading students.
- Access: Limited to sections they are assigned to teach. Read-only access to student profiles in their classes.

C. STUDENT

- Characteristics: End-users, varying technical literacy.

- Responsibilities: Registering for classes, viewing grades.

- Access: Restricted to their own data and public course schedules.

3.2 FUNCTIONAL REQUIREMENTS

The system must support the following functions:

- FR1: The system shall allow users to log in using a unique username and password.

- FR2: The system shall support the creation of courses with defined credits.

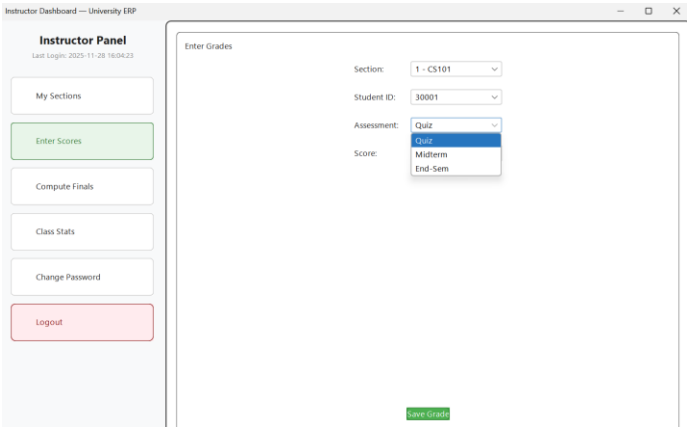
- FR3: The system shall allow admins to schedule sections, assigning instructors, rooms, and time slots.

- FR4: The system shall prevent students from registering for full sections or after deadlines.

- FR5: The system shall allow instructors to enter raw scores for Quizzes (20%), Midterms (30%), and End-Sem exams (50%).

- FR6: The system shall automatically compute the letter grade (A-F) based on the weighted total score.

- FR7: The system shall provide a maintenance mode switch that freezes all non-admin write operations.



3.3 NON-FUNCTIONAL REQUIREMENTS

- NFR1 (Security): All user passwords must be hashed before storage.

- NFR2 (Performance): Login and search operations should complete within 2 seconds.

- NFR3 (Reliability): The system must ensure data consistency (ACID properties) during enrollment transactions.

- NFR4 (Availability): The database should be hosted on a cloud provider (TiDB) to ensure high availability.

---

4. SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The application follows a Layered Architecture pattern, separating concerns into three distinct layers:

1. Presentation Layer (UI):
   - Handles user interaction via the console/desktop interface.
   - Captures input and displays formatted output.
   - Located in `edu.univ.erp.ui`.

2. Business Logic Layer (Service):
   - Contains the core functionality and rules.
   - Validates business rules (e.g., "Is the section full?", "Is maintenance mode on?").
   - Located in `edu.univ.erp.service`.

3. Data Access Layer (DAO):
   - Handles direct communication with the database.
   - Executes SQL queries and maps ResultSets to Java objects (Domain models).
   - Located in `edu.univ.erp.data`.

4.2 DATABASE DESIGN

The system utilizes two logical databases to separate authentication concerns from operational data.

4.2.1 AUTHENTICATION DATABASE (`auth_db`)

Table: `users_auth`

- `user_id` (INT, PK): Unique identifier.
- `username` (VARCHAR): Unique login name.
- `role` (ENUM): 'student', 'instructor', 'admin'.
- `password_hash` (VARCHAR): BCrypt hashed password string.
- `status` (ENUM): Account status (active/locked).
- `last_login` (TIMESTAMP): Audit trail for access.

4.2.2 ERP DATABASE (`erp_db`)

Table: `students`

- `user_id` (INT, PK, FK): Links to auth_db.
- `roll_no` (VARCHAR): Academic ID.
- `program` (VARCHAR): e.g., "Computer Science".
- `year` (INT): Current year of study.

Table: `instructors`

- `user_id` (INT, PK, FK): Links to auth_db.
- `department` (VARCHAR): e.g., "Physics".

Table: `courses`

- `course_code` (VARCHAR, PK): e.g., "CS101".

- `title` (VARCHAR): Course name.

- `credits` (INT): Credit weight.

Table: `sections`

- `section_id` (INT, PK): Unique instance ID.

- `course_code` (FK): The course being taught.

- `instructor_id` (FK): The teacher assigned.

- `capacity` (INT): Max students allowed.

- `day_time`, `room`: Scheduling details.

Table: `enrollments`

- `enrollment_id` (INT, PK): Unique record.

- `student_id` (FK): The student.

- `section_id` (FK): The section.

- `status` (ENUM): 'enrolled', 'dropped'.

- `final_grade` (VARCHAR): 'A', 'B', etc.

Table: `grades`

- `grade_id` (INT, PK).

- `enrollment_id` (FK).

- `component` (VARCHAR): 'Quiz', 'Midterm'.

- `score` (DECIMAL): The raw score.

4.3 CLASS DIAGRAM & COMPONENT DESIGN

The Java implementation mirrors the database structure with Domain classes (`Student`, `Course`, `Section`) acting as Data Transfer Objects (DTOs).

- `AdminService`: The "God class" for administrative tasks. It aggregates multiple DAOs (`StudentDAO`, `CourseDAO`, etc.) to perform complex transactions like creating a full user profile (which involves writing to both `auth_db` and `erp_db`).

- `StudentService`: Handles the logic for `registerForSection`. It implements the critical "Check-Then-Act" logic to prevent race conditions or rule violations (checking capacity, deadlines, and duplicates before inserting).

- `InstructorService`: Contains the grading logic. It encapsulates the weighted average formula, ensuring that grade calculation is consistent across the entire university.

---

5. IMPLEMENTATION DETAILS

5.1 TECHNOLOGY STACK

- Language: Java (JDK 17+)

- Database: MySQL / TiDB Cloud (Serverless SQL)

- Security Library: JBCrypt (for password hashing)

- Build Tool: Standard Java SDK tools / IDE support.

5.2 KEY MODULES AND SERVICES

A. Authentication Service (`AuthService`)

This service manages the login lifecycle. When a user attempts to log in, the service retrieves the stored hash from `users_auth`. It uses `BCrypt.checkpw()` to verify the input password against the hash. This ensures that plain-text passwords are never stored or compared in memory, mitigating credential theft risks.

B. Grading Logic (`InstructorService.computeAndPublishFinalGrade`)

The grading algorithm is hardcoded to ensure standardization:

- Quiz: 20% weight

- Midterm: 30% weight

- End-Semester Exam: 50% weight

The service fetches all raw scores for an enrollment, applies these weights, sums them up, and maps the percentage to a letter grade (A >= 90, B >= 80, etc.).

C. Maintenance Mode

A unique feature implemented in `SettingsDAO` and checked in every Service method. If the admin toggles "maintenance_on" to "true" in the `settings` table, all write operations (registration, grading) immediately fail with a maintenance error. This allows admins to safely perform backups or updates without data changing mid-process.

5.3 SECURITY IMPLEMENTATION

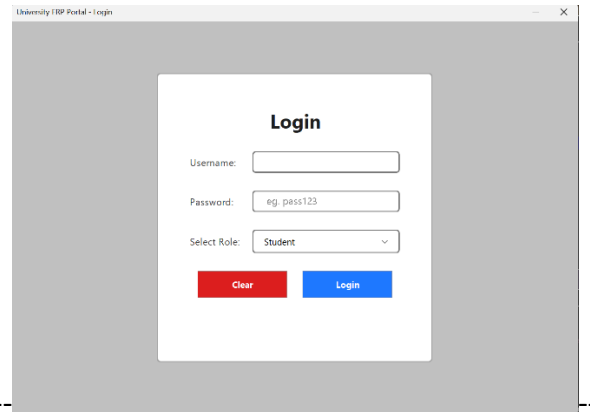
- SQL Injection Prevention: All Database Access Objects (DAOs) use `PreparedStatement` with parameterized queries. This completely neutralizes SQL injection attacks.

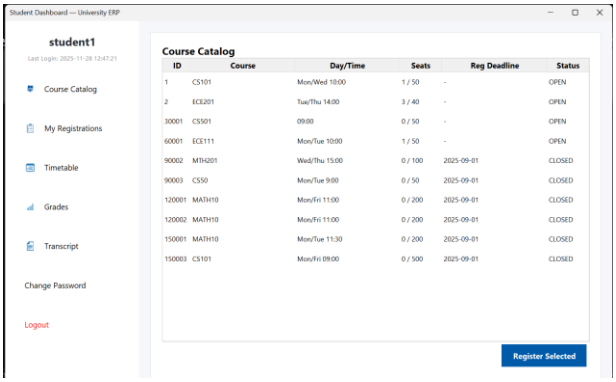
- Role-Based Access Control (RBAC): The UI layer checks the user's role object before displaying menus. For example, the "Create Course" option is simply not rendered for a Student user.

- Cloud Connectivity: The application connects to a remote TiDB cluster using SSL (`--ssl-mode=REQUIRED`), ensuring data in transit is encrypted.

---

6. USER INTERFACE AND USABILITY

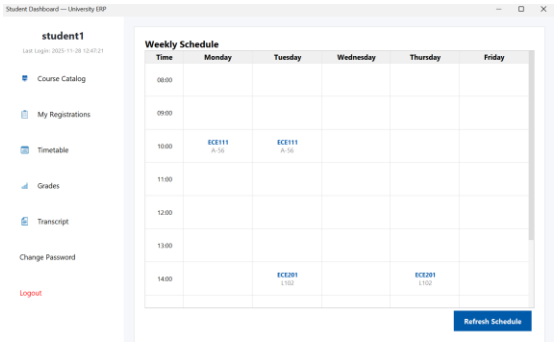




The application provides a structured Command Line Interface (CLI) or Desktop GUI (depending on the specific build configuration).

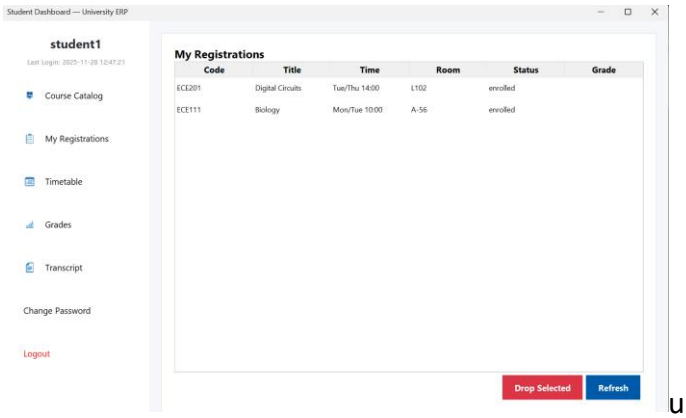
- Login Screen: The entry point requiring credentials.

- Dashboards: Upon successful login, users are routed to role-specific dashboards.



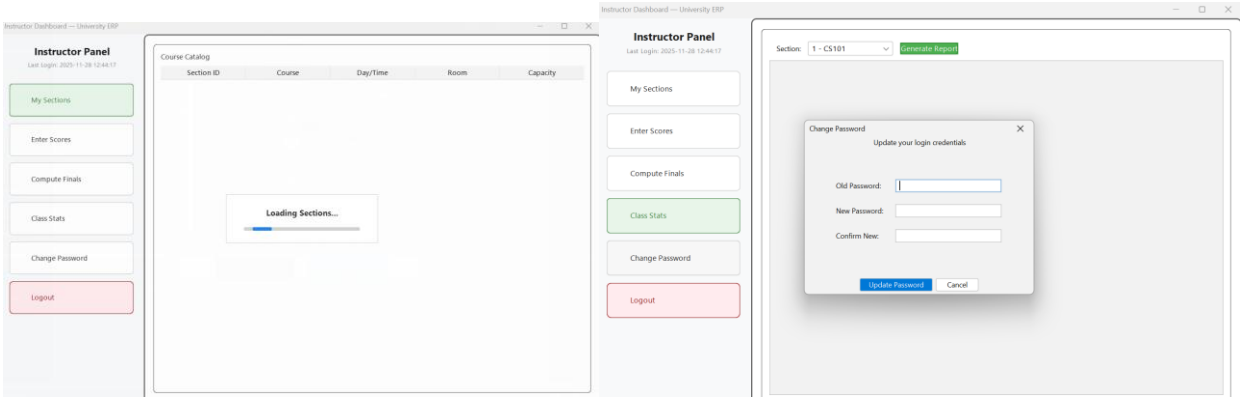
- Admin Dashboard: Options for "Manage Users", "Manage Courses", "System Settings".

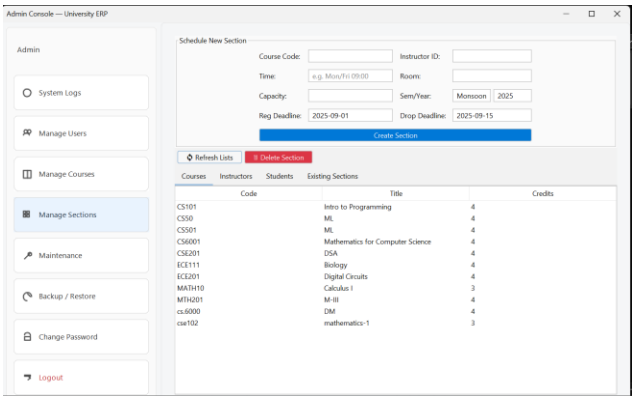
- Student Dashboard: Options for "My Sched



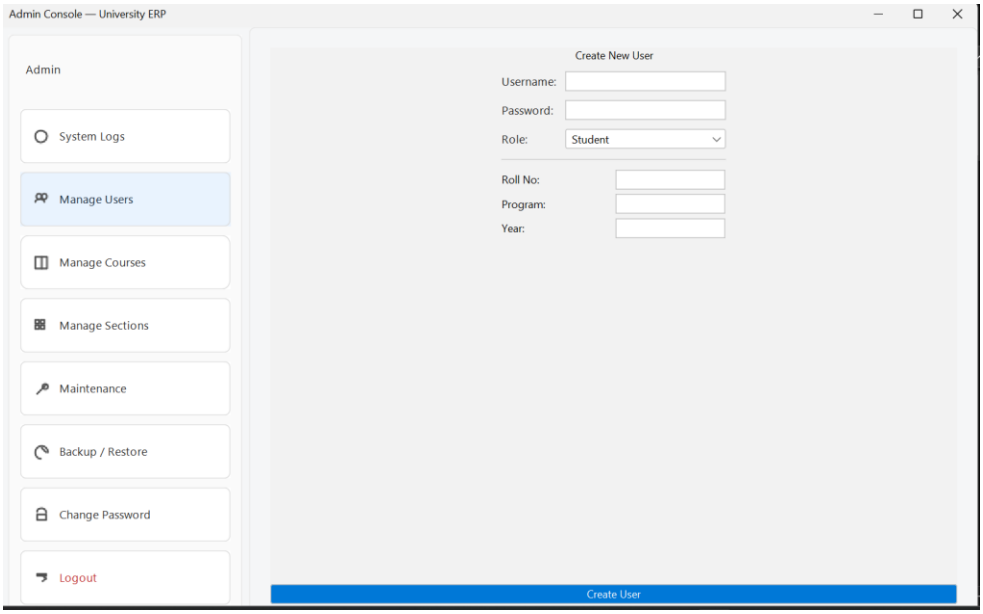
ule", "Register for Class", "View Transcript".

- Instructor Dashboard: Options for "My Classes", "Gradebook".





Feedback is immediate. Success messages are prefixed with "Success:", and error messages with "Error:", allowing users to quickly understand the outcome of their actions.



---

7. TESTING AND VALIDATION

Quality Assurance was integral to the development process.

7.1 UNIT TESTING

JUnit was used to test individual service methods in isolation.

- `AdminServiceTest`: Verifies that creating a user correctly inserts records into the database mock.

- `StudentServiceTest`: Crucial tests for the registration logic. Test cases included:

  - Attempting to register for a full class (Expected: Error).

  - Attempting to register after the deadline (Expected: Error).

  - Attempting to register for a class already taken (Expected: Error).

7.2 INTEGRATION TESTING

Manual integration tests were performed to verify the flow of data between the application and the cloud database.

- Scenario: An admin creates a course -> An instructor is assigned -> A student registers -> The instructor grades the student.

- Result: Verified that the final grade appeared correctly on the student's transcript view.

---

8. LIMITATIONS AND FUTURE ENHANCEMENTS

8.1 LIMITATIONS

- Single-Threaded UI: The current console/desktop implementation processes requests synchronously.

- Hardcoded Grading Scale: The grading weights (20/30/50) are currently hardcoded in the Java logic rather than being configurable per course.

8.2 FUTURE ENHANCEMENTS

- Dynamic Grading Schemes: Allow instructors to define their own weighting schemes for different courses.

- Waitlist Functionality: Implement a queue system for full sections, automatically enrolling students if a spot opens up.

- Web Interface: Port the frontend to a React/Angular web application to allow access from any device.

- Notification System: Email or SMS alerts for grade publishing or deadline reminders.

---

9. CONCLUSION

The University ERP System successfully meets the core objectives of digitizing the academic administration process. By leveraging a robust Java backend and a scalable cloud database, the system provides a secure, reliable, and efficient platform for managing the complex relationships between students, courses, and instructors.

The implementation of strict business rules for enrollment and grading ensures fairness and accuracy, while the security measures protect sensitive user data. While there is room for future expansion, particularly in the areas of UI modernization and dynamic configuration, the current system stands as a solid foundation for the university's digital infrastructure.